```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np
```

```
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    "Images/train",
    image_size = (224, 224),
    batch_size = 32,
    seed = 42
)
```

```
Found 60 files belonging to 5 classes.
```

```
print(train_ds.cardinality().numpy())   # number of batches
```

```
2
```

```
count = sum(len(x) for x, y in train_ds)
print(count)   # should print 15
```

```
60
```

```
class_names = train_ds.class_names
print(class_names)
```

```
['Blight', 'Healthy', 'Leaf_spot', 'Mildew', 'Rust']
```

```
import tensorflow as tf
import os

# Image settings
img_size = (224, 224)
batch = 32
data_dir = "/content/Images"

# Load training images and split them into training and validation sets
train_ds = tf.keras.utils.image_dataset_from_directory(
    os.path.join(data_dir, "train"),
    validation_split=0.2,
    subset="training",
    seed=42,
    image_size=img_size,
    batch_size=batch
)

# Load validation images from the same training folder
val_ds = tf.keras.utils.image_dataset_from_directory(
    os.path.join(data_dir, "train"),
    validation_split=0.2,
    subset="validation",
    seed=42,
    image_size=img_size,
    batch_size=batch,
    shuffle=False
)

# Get and display class names
class_names = train_ds.class_names
print("Class names:", class_names)

# Improve data-loading performance
AUTOTUNE = tf.data.AUTOTUNE

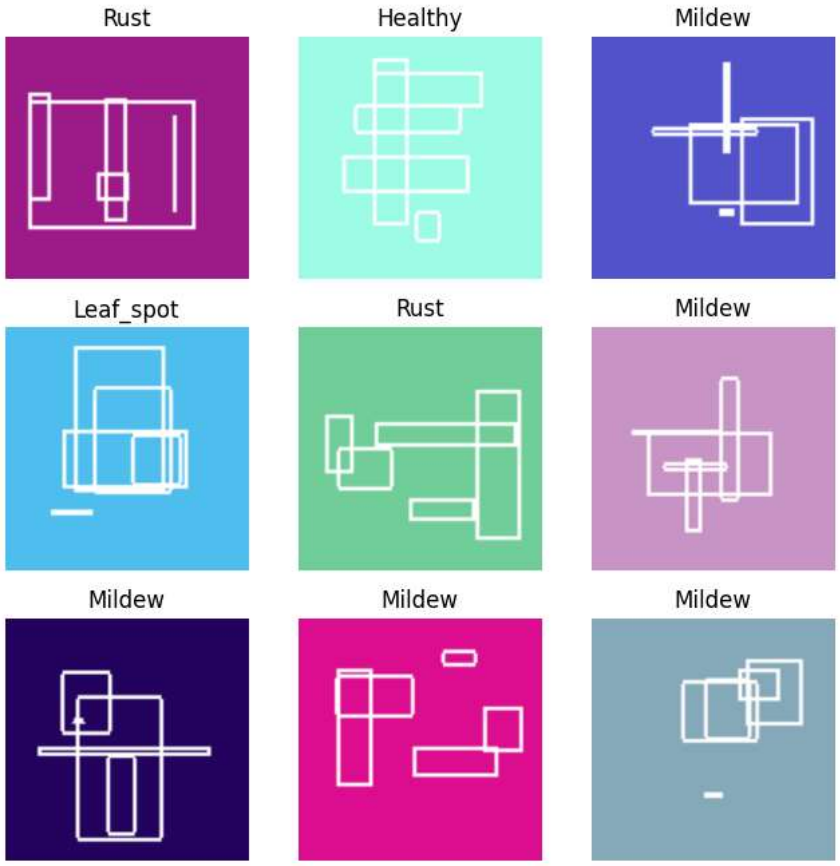
train_ds = train_ds.prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.prefetch(buffer_size=AUTOTUNE)

print("Training and validation datasets loaded successfully!")
```

```
Found 60 files belonging to 5 classes.
Using 48 files for training.
```

```
Found 60 files belonging to 5 classes.
Using 12 files for validation.
Class names: ['Blight', 'Healthy', 'Leaf_spot', 'Mildew', 'Rust']
Training and validation datasets loaded successfully!
```

```
plt.figure(figsize=(8, 8))
for images, labels in train_ds.take(1):
    for i in range(min(9, len(images))):
        plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")
plt.show()
```




```
from sklearn.metrics import classification_report, confusion_matrix
```

```
Cnn = models.Sequential([
    layers.Input(shape=(224, 224, 3)),
    layers.Rescaling(1./255),               # normalization

    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(len(class_names), activation="softmax")
])

Cnn.compile(optimizer="adam",
            loss="sparse_categorical_crossentropy",
            metrics=["accuracy"])

Cnn.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 224, 224, 3)	0
conv2d (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
flatten (Flatten)	(None, 186624)	0
dense (Dense)	(None, 64)	11,944,000
dense_1 (Dense)	(None, 5)	325

Total params: 11,963,717 (45.64 MB)
Trainable params: 11,963,717 (45.64 MB)

```
history = Cnn.fit(train_ds, validation_data=val_ds, epochs=10)
```

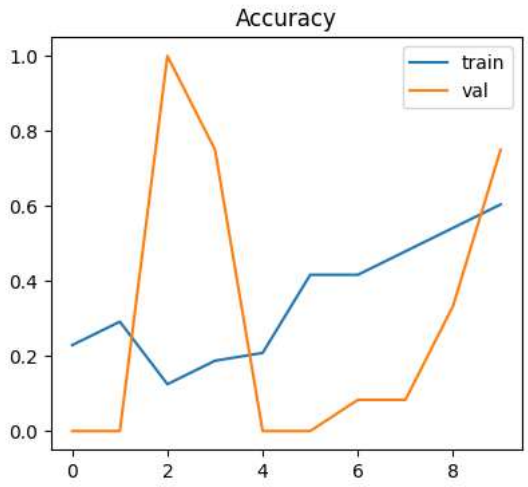
```
Epoch 1/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 7s 2s/step - accuracy: 0.2292 - loss: 6.9468 - val_accuracy: 0.0000e+00 - val_loss: 4.1666
Epoch 2/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - accuracy: 0.2917 - loss: 4.4588 - val_accuracy: 0.0000e+00 - val_loss: 1.7606
Epoch 3/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 1s/step - accuracy: 0.1250 - loss: 2.6685 - val_accuracy: 1.0000 - val_loss: 0.5362
Epoch 4/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 6s 2s/step - accuracy: 0.1875 - loss: 1.8286 - val_accuracy: 0.7500 - val_loss: 1.0285
Epoch 5/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - accuracy: 0.2083 - loss: 1.4978 - val_accuracy: 0.0000e+00 - val_loss: 1.7515
Epoch 6/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 1s/step - accuracy: 0.4167 - loss: 1.4014 - val_accuracy: 0.0000e+00 - val_loss: 1.7572
Epoch 7/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 6s 1s/step - accuracy: 0.4167 - loss: 1.2989 - val_accuracy: 0.0833 - val_loss: 1.5300
Epoch 8/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - accuracy: 0.4792 - loss: 1.2072 - val_accuracy: 0.0833 - val_loss: 1.5906
Epoch 9/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 6s 3s/step - accuracy: 0.5417 - loss: 1.1342 - val_accuracy: 0.3333 - val_loss: 1.1524
Epoch 10/10
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - accuracy: 0.6042 - loss: 1.0177 - val_accuracy: 0.7500 - val_loss: 1.1899
```

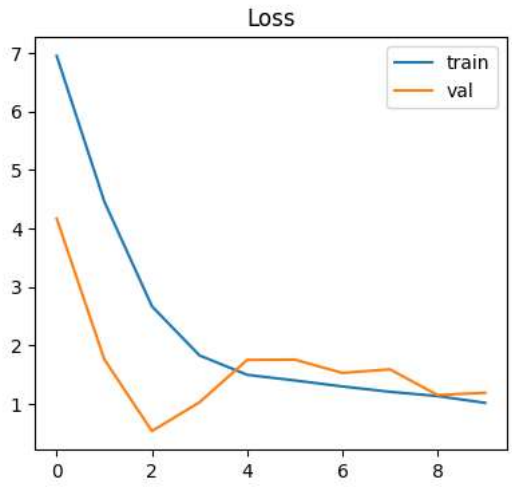
```
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="train")
plt.plot(history.history["val_accuracy"], label="val")
plt.title("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="train")
plt.plot(history.history["val_loss"], label="val")
plt.title("Loss")
plt.legend()

plt.show()
```






```
Cnn.evaluate(val_ds)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 459ms/step - accuracy: 0.7500 - loss: 1.1899
[1.1898738145828247, 0.75]
```

```
y_true = np.concatenate([y.numpy() for x, y in val_ds])
y_pred = Cnn.predict(val_ds)
y_pred_classes = np.argmax(y_pred, axis=1)

print(classification_report(
    y_true, y_pred_classes,
    labels=range(len(class_names)),
    target_names=class_names,
    zero_division=0))

cm = confusion_matrix(y_true, y_pred_classes, labels=range(len(class_names)))
plt.figure(figsize=(6, 5))
plt.imshow(cm, cmap="Blues")
plt.xticks(range(len(class_names)), class_names, rotation=45)
plt.yticks(range(len(class_names)), class_names)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 300ms/step
              precision    recall  f1-score   support

      Blight       0.00      0.00      0.00         0
     Healthy       0.00      0.00      0.00         0
   Leaf_spot       0.00      0.00      0.00         0
      Mildew       0.00      0.00      0.00         0
        Rust       1.00      0.75      0.86        12
```

```
for images, labels in val_ds.take(1):
    preds = Cnn.predict(images)
    for i in range(min(3, len(images))):
        plt.figure(figsize=(3, 3))
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(f"Actual: {class_names[labels[i]]}\nPredicted: {class_names[np.argmax(preds[i])]}")
        plt.axis("off")
        plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 339ms/step
```

